

M10 Review / Repair - Teaching RC2

OPEN ONLY AFTER SAVED ATTEMPT

Use surgically

Read only the competency you missed. Close this file and solve a changed retry. Reading review is not mastery.

P01 repair - Why containers exist

"Works on my machine" often means hidden OS/package/runtime differences. Containers package an application filesystem/runtime/config boundary so the same image can run consistently.

Core repair rule: A container is an isolated process/filesystem/network namespace environment built from an image; it is not a full separate hardware machine.

Proof: volume/external DB -> durable state outside container writable layer

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P02 repair - Images and containers

Docker debugging becomes much easier when you know whether the problem is build-time image content or runtime container state.

Core repair rule: docker image ls shows local images; docker ps shows running containers; docker ps -a includes stopped.

Proof: docker rmi -> remove image if not required

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P03 repair - Writing a Dockerfile

A Dockerfile is part of the production artifact. Small mistakes can leak secrets, bloat images or make builds non-reproducible.

Core repair rule: FROM selects a base image; pin/choose deliberately.

Proof: command -> pipeline.py

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P04 repair - Ports

Port confusion is one of the most common beginner Docker bugs: localhost means different things depending on which process/container you are in.

Core repair rule: A process must listen on a container port; Docker port publishing maps host_port:container_port.

Proof: db container itself -> localhost:5432 if server listens there

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P05 repair - Volumes and persistence

If a database container is recreated without a volume, its writable-layer data may vanish. If source code is baked/mounted incorrectly, runtime may not see expected files.

Core repair rule: Container writable layer is tied to the container lifecycle.

Proof: bind mount -> specific host directory shared into container

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P06 repair - Networks and service communication

Multiple services may start at different times. A container being "started" does not mean the database is ready to accept connections.

Core repair rule: Compose creates a default network where services resolve each other by service name.

Proof: application success -> its own work completed

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P07 repair - Compose, configuration and secrets

Container reproducibility depends on explicit configuration. Secret handling is part of that contract, not an afterthought.

Core repair rule: Compose describes services, build/image, environment, networks, volumes, secrets, healthchecks and dependencies.

Proof: image layers -> no password value

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P08 repair - Why orchestration exists

Production data jobs have dependencies, schedules, retries, backfills, logs and state across many tasks. Orchestrators coordinate these responsibilities.

Core repair rule: Orchestration defines task dependencies and when logical data intervals should be processed.

Proof: Docker -> runtime packaging

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P09 repair - Airflow architecture and UI

Operational debugging depends on knowing whether the DAG parsed, scheduler created a run, task was queued/running/failed and what its log says.

Core repair rule: Airflow 3 training image in this course uses apache/airflow:3.3.0 and TaskFlow imports from airflow.sdk.

Proof: cause -> task log traceback/message

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P10 repair - DAGs, tasks and dependencies

Task boundaries decide what can retry independently and what operational state Airflow exposes.

Core repair rule: A DAG must be acyclic; dependencies form a directed acyclic graph.

Proof: load -> idempotent durable publication

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.